DT081A Signal and Image Processing
Lab 2

Introduction

The theory related to this lab is found in lectures 4–7.

All the assignments of this lab will be carried out on a computer. You will be considering digital signals. The sampling frequency of these signals will be large enough so that no aliasing occurs (i.e., $f_s > 2 \times \max(f)$). Amplitude quantization will be such that it can be considered continuous. This means that the theory for continuous signals applies approximately.

First, set up your Python environment to begin signal processing.

1 Time-Domain Analysis: Superposition and Windowing

Create a time vector corresponding to the time instances we consider:

```
import numpy as np
import matplotlib.pyplot as plt

fs = 250  # Sampling frequency (Hz)
t = np.arange(0, 0.6, 1/fs)

# Generate a signal consisting of three sinusoids
A1, A2, A3 = 1, 0.1, 0.3  # Amplitudes
f1, f2, f3 = 50, 55, 120  # Frequencies in Hz
ph1, ph2, ph3 = 0, np.pi/4, np.pi/2  # Phases in radians

x = A1 * np.cos(2 * np.pi * f1 * t + ph1) + A2 * np.cos(2 *
    np.pi * f2 * t + ph2) + A3 * np.cos(2 * np.pi * f3 * t +
    ph3)

# Plot the signal
plt.plot(t, x)
plt.title('Time-Domain Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid()
```

```
plt.show()
```

Now compute the signal's Fourier transform and its amplitude spectrum:

```
from scipy.fft import fft, fftfreq

# Compute the Fourier transform and the Amplitude spectrum
X = fft(x)
A_x = np.abs(X)
f = fftfreq(len(x), 1/fs)

# Plot the amplitude spectrum
plt.plot(f, A_x)
plt.title('Amplitude Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

Question: Why does the amplitude spectrum contain negative frequencies?

In the case of a real-valued signal, $x(t) \in \mathbb{R}$, the Fourier transform $X(f)$ contains the same information for negative frequencies. Therefore, let us remove the negative frequencies from the graph:

```
# Plot the amplitude spectrum for positive frequencies
plt.plot(f[:len(f)//2], A_x[:len(f)//2])
plt.title('Amplitude Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

2 Frequency Identification

The signal in the example above $x(t)$ is actually defined for all times from $-\infty$ to ∞, but you only generated data for 0.4 seconds. This means that you have applied a rectangular window. Now, apply two other windows (Hann and Hamming) and compute their amplitude spectra.

```
from scipy.signal import windows

# Apply Hann and Hamming windows
x_hn = x * windows.hann(len(x))
x_hm = x * windows.hamming(len(x))

# Compute Fourier transform for windowed signals
X_hn = fft(x_hn)
X_hm = fft(x_hm)
```

```
# Plot amplitude spectrum for both windows
# Reuse the code from Section 2
```

Question: Which of the windows (rectangular, Hann, and Hamming) is best able to separate frequencies close to each other? Why?

3 Linear Time-Invariant Systems

3.1 System 1

Simulate a linear time-invariant system using a convolution sum for a finite impulse response (FIR) system.

```
# Define the impulse response
h = [1, 0.5, 0.3, 0.1, -0.1, -0.4, -0.2, 0.1]

# Simulate the system using convolution
impulse_response = np.convolve(h, [1] + [0]*(len(x)-1))

# Plot the impulse response
plt.stem(impulse_response, use_line_collection=True)
plt.title('Impulse Response')
plt.xlabel('Samples')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

Questions:

- Does the convolution give the correct response?
- Is this a FIR or IIR system? How can you tell?

3.2 System 2

Another system is defined by the following difference equation

$$y(n) = b_0 \cdot x(n) + b_1 \cdot x(n-1) - a_1 y(n-1) - a_2 \cdot y(n-1)$$

where $b_0 = 1$, $b_1 = 0.3$, $a_1 = 0.6$, and $a_2 = -0.35$.

Questions:

- Write a small program that calculates the output from this LTI system by iteratively applying this equation, i.e. make a loop. Let the input be an impulse vector of length `len(x)`, `x` being the vector from Section 1. Save the output in a vector equally long as the input vector. Assume all values of input and output to be zero for $n < 0$. Plot the output vector. Let the input be a unit pulse and use a total of 100 samples for the input.

- Is this a FIR or IIR system? How can you tell?

Simulate the same system using a `filter` function.

```
# Simulate by using the filter implementation
from scipy.signal import lfilter

# Re-use the values for the filter taps from above
a = [1, 0.6, -0.35]
b = [1, 0.3]

# Impulse response
y2 = lfilter(b,a,[1] + [0]*(len(x)-1))

# Plot
plt.stem(y2)
plt.title('Output of the LTI System')
plt.xlabel('Sample Index')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

Questions:

- Do you get the same results in the two simulations?

4 Frequency-Domain Analysis

Regenerate the signal from the first task and compute its amplitude and phase spectrum.

```
# Generate a signal consisting of three sinusoids
fs = 250  # Sampling frequency (Hz)
t = np.arange(0, 0.6, 1/fs)
A1, A2, A3 = 1, 0.1, 0.3  # Amplitudes
f1, f2, f3 = 50, 55, 120  # Frequencies in Hz
ph1, ph2, ph3 = 0, np.pi/4, np.pi/2  # Phases in radians

x = A1 * np.cos(2 * np.pi * f1 * t + ph1) + A2 * np.cos(2 *
    np.pi * f2 * t + ph2) + A3 * np.cos(2 * np.pi * f3 * t +
    ph3)

# Compute amplitude spectrum and phase spectrum
X = fft(x)
A_x = np.abs(X) # amplitude spectrum
phase_x = np.angle(X) # phase spectrum
f = fftfreq(len(x), 1/fs) # frequency vector

# Plot amplitude and phase spectrum
plt.subplot(1, 2, 1)
```

```
plt.plot(f[:len(f)//2], A_x[:len(f)//2])
plt.grid()
plt.title('Amplitude Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')

plt.subplot(1, 2, 2)
plt.plot(f[:len(f)//2], phase_x[:len(f)//2])
plt.title('Phase Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Phase (radians)')
plt.show()
```

Question: From these spectra, decide the frequencies, amplitudes, and phases of the sinusoids in the signal. Use the zoom-functionality in the graphs to see better. Do the amplitudes and phases agree with what you have defined in the code?

5 System Frequency Response

Here you shall investigate the consequence of a system by studying its System Frequeny response. You shall study System 2 given above. Start by retrieving the frequency response by calculating the Fourier transform of its impulse response (which you already have computed). Then plot the Amplification and Phase shift as functions of frequency.

```
# Compute the System Frequency response
h = lfilter(b,a,[1]+[0]*(len(x)-1))
H = fft(h)

# Plot the System Amplification and Phaseshift
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(f[:len(f)//2], np.abs(H[:len(f)//2]))
plt.title('System Amplification')
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(f[:len(f)//2], np.angle(H[:len(f)//2]))
plt.title('System Phase shift')
plt.grid()
plt.show()
```

Now run a single sinusoidal signal through this system.

```
# Single sinusoidal signal
x2 = A1 * np.cos(2 * np.pi * f1 * t + ph1)

# Run the signal through the system
```

```
y3 = lfilter (b ,a , x2)

# Compute amplitude spectrum and phase spectrum
Y3 = fft(y3)
A_y = np.abs(Y3) # amplitude spectrum
phase_y = np.angle(Y3) # phase spectrum

# Plot amplitude and phase spectrum
# User similar code as in Section 4
```

Questions:

- Identify the amplitude and phase of both input and output signals by analysing them in the frequency domain as in section 4.
- Analyse also the system frequency response (in frequency domain). Verify the relation between output and input $Y(f) = H(f)X(f)$ for both amplitude and phase for this particular input and output signals.

6 Filtering

Digital filters can easily be designed with computer support. There are a number of different "standard" filters with different characteristics. E.g. a Butterworth filter is maximum flat filter in the amplification. Other standard filters are Chebyshev (type 1 and type 2), Elliptic (Cauer), and Bessel. Originally these methods are for designing time-continuous (analogue) filters, but can also be transformed to discrete-time (digital) filters. Programs make these transformations automatically. The filters are designed by giving design-type (butter/cheby1, etc.), type of filter (LP/HP/BP/BS), cut-off frequency(ies) f_c, sampling frequency f_s, and filter order N. The filter order gives how many "taps" a discrete-time filter contains. A higher order implies a sharper transition between the pass and stop bands, but also implies longer time-delays in the filters.

Next, you shall design Butterworth and Chebyshev (type 1) filters. Try to different designs: both LP and HP, different cut-off frequencies, and different orders. Plot the amplification and phase shift for the filters to compare and understand what happens when you change the parameters.

```
from scipy import signal

fs1 = 2500 # [Hz] sampling frequency
fc = 200  # [Hz] cut-off frequency
N = 5     # order
Rp = 3    # [dB] ripple

# Low-pass Butterworth filter
b_b, a_b = signal.butter(N, fc, btype='low', fs=fs1)
```

```
# Low-pass Chebyshev 1 filter
b_c1, a_c1 = signal.cheby1(N, Rp, fc, btype='low', fs=fs1)

# Plot the System Amplification and Phaseshift
H_b = fft(lfilter(b_b,a_b,[1]+[0]*(len(x)-1)))
H_c1 = fft(lfilter(b_c1,a_c1,[1]+[0]*(len(x)-1)))

plt.figure(12,figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(f[:len(f)//2], 20 * np.log10(np.abs(H_b[:len(f)
    //2])), label='Butterworth')
plt.plot(f[:len(f)//2], 20 * np.log10(np.abs(H_c1[:len(f)
    //2])), label='Chebyshev 1')
plt.title('System Amplification')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude [dB]')
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(f[:len(f)//2], np.angle(H_b[:len(f)//2]), label='
    Butterworth')
plt.plot(f[:len(f)//2], np.angle(H_c1[:len(f)//2]), label='
    Chebyshev 1')
plt.title('System Phase shift')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude [dB]')
plt.legend()
plt.grid()
plt.show()
```

Questions:

- How do Butterworth and Chebyshev type 1 differ?
- How do the filters differ in amplification and phase shift when changing LP/HP, f_c, order N.
- Write the python code to design BP and BS filters. (Search the web or ask ChatGPT.)

7 Frequency-Time Domain Analysis

You shall now record your own voice and analyse it in frequency domain and how it evolves over time. Start by recording your voice by saying your own name, e.g. "Kalle", using the web-based *Online Voice Recorder* (https://online-voice-recorder.com/). Save the recording using the `.wav` format to your computer.

The python code below loads the data for further processing. The recording $x_{\text{voice}}(t)$ is subdivided into sub-sequences by changing the vector into a

matrix $x_{\text{voice}}(k,t)$, where each row k (column?) corresponds to one of these sub-sequences. The program computes the Fourier transform for each of these sub-sequences, $X_{\text{voice}}(k,f)$. The code also computes their power spectrum $|X|^2_{\text{voice}}(k,f)$

```
# Load the voice recording and transform to Fourier-Time
    domain.
import scipy.io.wavfile as wav

# Load .wav file
fs, x_voice = wav.read('path_to_your_wav_file.wav')
# For windows:
#fs, x_voice = wav.read(r'path_to_your_wav_file.wav')

# Normalize the signal if it's stereo (2 channels)
if len(x_voice.shape) == 2: x_voice = np.mean(x_voice, axis
    =1)

# Parameters
k = 1024  # Number of samples per sub-sequence
n = len(x_voice) // k  # Number of sub-sequences
xm_xvoice = x_voice[:n*k].reshape(n, k)  # Reshape into
    matrix

# Compute Fourier transform for each sub-sequence
Xvoice = np.fft.fft(xm_xvoice, axis=1)

# Compute power spectrum |Xvoice(k, f)|^2
power_spectrum = np.abs(Xvoice) ** 2

# Define the frequency axis
frequencies = np.fft.fftfreq(k, d=1.0/fs)

# Define time axis (in seconds)
time_axis = np.arange(0, n) * (k / fs)

# Plot the power spectrum
plt.figure(13, figsize=(10, 6))
plt.imshow(10 * np.log10(power_spectrum.T + 1e-12), origin='
    lower', aspect='auto',
           extent=[time_axis[0], time_axis[-1], frequencies
               [0], frequencies[k // 2]], cmap='viridis')
plt.colorbar(label='Power (dB)')
plt.xlabel('Time (seconds)')
plt.ylabel('Frequency (Hz)')
plt.title('Power Spectrum $|Xvoice(t,f)|^2$')
plt.show()
```

Questions:

- What is $X(t,f)$ called? What is $|X(t,f)|^2$ called?

- How does changing the length of the sub-sequences affect time and frequency resolution? Try out different lengths by changing `k` in the code.
- Try to identify the letters of your name by looking at $|X(t, f)|^2$.